

Project_Dataanalysis

January 13, 2026

Does having Ballon d'Or players increase a nation's chance of winning the World Cup, or does winning the World Cup increase a player's chance of receiving Ballon d'Or votes?

Ballon d'Or & World Cup – Datenanalyse

```
[161]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import seaborn as sns
from statsmodels.graphics.mosaicplot import mosaic

from scipy.stats import mannwhitneyu
from scipy.stats import fisher_exact
```

Load data from csv files

```
[162]: players = pd.read_csv("players.csv")
ballondor = pd.read_csv("ballondor_all_ranked.csv")
standing = pd.read_csv("tournament_standings_2.csv")
matches_players = pd.read_csv("player_appearances.csv")
```

Show the first 10 entries of each tabel

```
[163]: #players.head()
```

```
[164]: #ballondor_combined.head()
```

```
[165]: #standing.head()
```

```
[166]: #matches_players.head()
```

Filter for Men's worldcup due to tabel containing Womens worldcup as well

Filter out the woldcup years

Converting "Years" of the ballondor set into an integer

Print out the Number of Woldrcups considered in this analysis

```
[167]: wc_standings = standing[standing["tournament_name"].str.contains("FIFA Men's_
↳World Cup", case=False, na=False)].copy()
wc_matches_players = matches_players[matches_players["tournament_name"].str.
↳contains("FIFA Men's World Cup", case=False, na=False)].copy()

wc_standings["year"] = wc_standings["tournament_name"].str.extract(r"(\d{4})").
↳astype(int)
wc_matches_players["year"] = wc_matches_players["tournament_name"].str.
↳extract(r"(\d{4})").astype(int)

ballondor["Year"] = ballondor["Year"].astype(int)

print(f"Number of World Cups we consider: ")
print((len(wc_standings)/4))
```

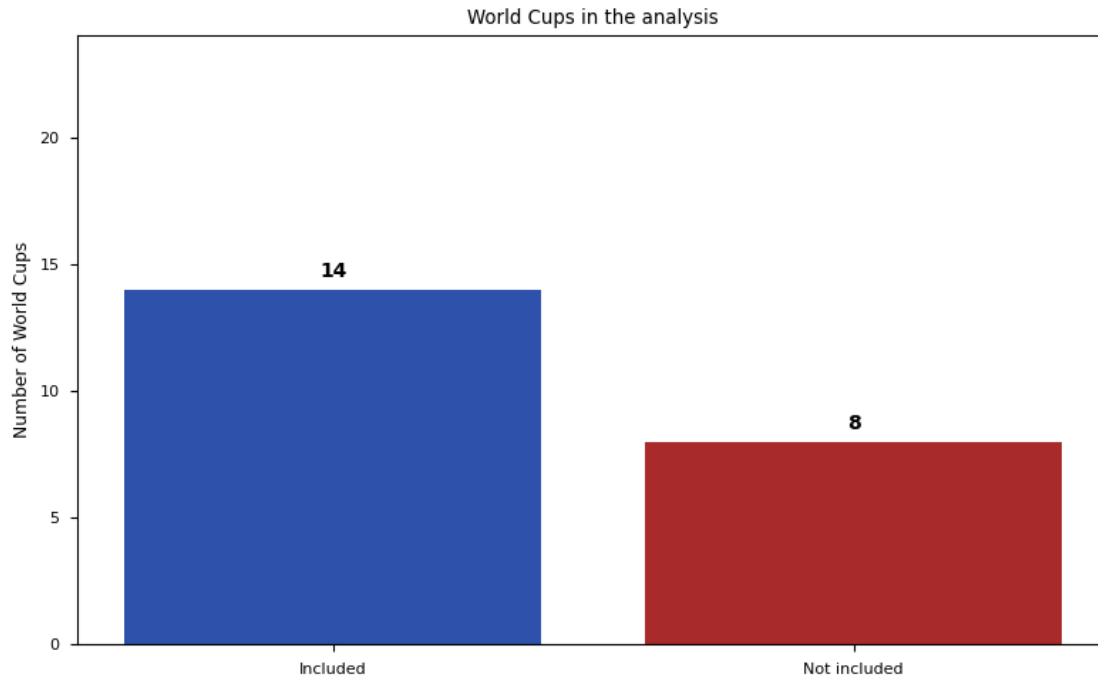
Number of World Cups we consider:
14.0

```
[168]: total_wcs = len(wc_standings) / 4 # 14
all_wcs_ever = 22 # total number of World Cups

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(['Included', 'Not included'],
      [total_wcs, all_wcs_ever - total_wcs],
      color=["#2E51AB", "#A92A2A"])
ax.set_ylabel('Number of World Cups')
ax.set_title('World Cups in the analysis')
ax.set_ylim(0, all_wcs_ever + 2)

for i, v in enumerate([total_wcs, all_wcs_ever - total_wcs]):
    ax.text(i, v + 0.5, str(int(v)), ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



```
[169]: unique_years = sorted(wc_standings["year"].unique())
all_wc_years = list(range(1930, 2023, 4)) #all world cups from 1930 to today in 4
      ↪ year steps

fig, ax = plt.subplots(figsize=(14, 4))

#Included world cups
ax.scatter(
    unique_years,
    [1] * len(unique_years),
    s=200,
    c='#2E86AB',
    label='Included in analysis',
    zorder=3
)

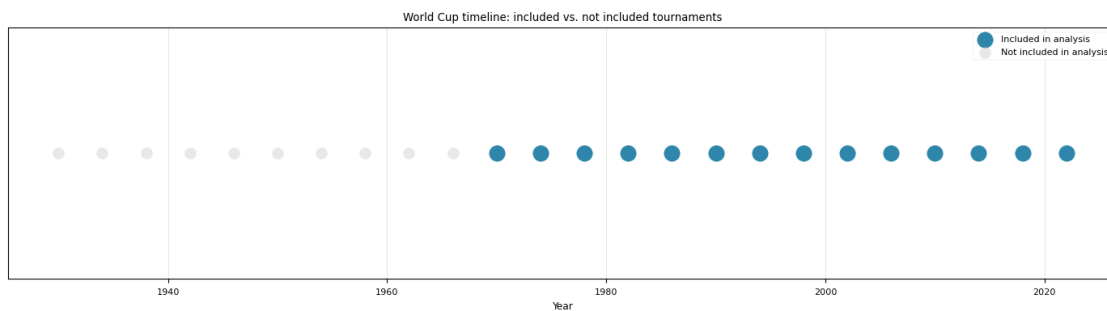
#Not included world cups
not_included_years = [y for y in all_wc_years if y not in unique_years]
ax.scatter(
    not_included_years,
    [1] * len(not_included_years),
    s=100,
    c='lightgray',
    label='Not included in analysis',
```

```

        alpha=0.5,
        zorder=2
    )
    ax.set_yticks([])
    ax.set_xlabel('Year')
    ax.set_title('World Cup timeline: included vs. not included tournaments')
    ax.legend()
    ax.grid(axis='x', alpha=0.3)
    plt.tight_layout()
    plt.show()

    print(f"Years included in analysis: {[int(year) for year in unique_years]}")
    print(f"Number of World Cups considered in total: {len(unique_years)}")

```



Years included in analysis: [1970, 1974, 1978, 1982, 1986, 1990, 1994, 1998, 2002, 2006, 2010, 2014, 2018, 2022]

Number of World Cups considered in total: 14

Just printing the years of the cup with the amount of teams in the standing tabel (always 4 because standing.csv has 4 entries for each tournament)

```
[170]: #wc_standings["year"].value_counts().sort_index()
```

Unify the player names of both list wc_matches_players and ballondor. For wc_matches_player append both names together.

```
[171]: wc_matches_players["player_fullname"] = (
        wc_matches_players["given_name"].fillna("") + " " +
        wc_matches_players["family_name"].fillna("")
    ).str.strip().str.lower()

    ballondor["player_fullname"] = ballondor["Player"].str.strip().str.lower()

```

Print the first 10 entries of both tabel columns "player_fullname"

```
[172]: print(ballondor["player_fullname"].head(10))
print(wc_matches_players["player_fullname"].head(10))
```

```
0      gerd müller
1      bobby moore
2      gigi riva
3  franz beckenbauer
4      wolfgang overath
5      dragan dzajic
6      johan cryuff
7      gordon banks
8      sandro mazzola
9      rinus israel
Name: player_fullname, dtype: object
0      ignacio calderón
1      gustavo peña
2      mario perez
3      guillermo hernández
4  horacio lópez salgado
5      josé vantolrá
6      javier guzmán
7      héctor pulido
8      mario velarde
9      javier valdivia
Name: player_fullname, dtype: object

Check for duplicates
```

```
[173]: wc_matches_players["player_fullname"].nunique(), len(wc_matches_players)
```

```
[173]: (5079, 20618)
```

```
[174]: ballondor["player_fullname"].nunique(), len(ballondor)
```

```
[174]: (389, 810)
```

Get rid of duplicates and make every entry for player in tournament unique. Important: One player has more than one appearance in the same WC tournament. We only need one of his entries and not all so make it unique

```
[175]: unique_players = (
    wc_matches_players
    .groupby(["year", "player_id", "player_fullname", "team_id", "team_name"],
    ↪as_index=False)
    .size()
)

unique_players.head()
```

```
[175]:   year player_id      player_fullname team_id      team_name  size
0  1970   P-00086      roberto rivas    T-27    El Salvador    3
1  1970   P-00119        gigi riva    T-41         Italy    6
2  1970   P-00746      tom turesson    T-74        Sweden    2
3  1970   P-01172    dobromir zhechev    T-10      Bulgaria    2
4  1970   P-02102  anatoliy byshovets    T-72  Soviet Union    4
```

Merge WC players and ballondor candidates of the same year. With this merge we link each WC Player with the ballondor ranking of that same year Resulting tabel gives information, if the player has been bd candidate or winner in the year of the wc and the year and rank.

```
[176]: players_bd_same = unique_players.merge(
    ballondor[["Year", "player_fullname", "Rank"]],
    how="left",
    left_on=["year", "player_fullname"],
    right_on=["Year", "player_fullname"],
    suffixes=("", "_bd_same")
)

players_bd_same["is_bd_candidate_same_year"] = (players_bd_same["Rank"] <= 15).
↳astype(int) #3
players_bd_same["is_bd_winner_same_year"] = (players_bd_same["Rank"] == 1).
↳astype(int)

players_bd_same.head(10)
```

```
[176]:   year player_id      player_fullname team_id      team_name  size  Year \
0  1970   P-00086      roberto rivas    T-27    El Salvador    3    NaN
1  1970   P-00119        gigi riva    T-41         Italy    6  1970.0
2  1970   P-00746      tom turesson    T-74        Sweden    2    NaN
3  1970   P-01172    dobromir zhechev    T-10      Bulgaria    2    NaN
4  1970   P-02102  anatoliy byshovets    T-72  Soviet Union    4    NaN
5  1970   P-02166    josé quintanilla    T-27    El Salvador    2    NaN
6  1970   P-02372      georgi popov    T-10      Bulgaria    2    NaN
7  1970   P-02403    emerich dembrovski    T-61      Romania    3    NaN
8  1970   P-02662      oscar zubia    T-84      Uruguay    2    NaN
9  1970   P-02965  not applicable piazza    T-09      Brazil    6    NaN
```

```
Rank  is_bd_candidate_same_year  is_bd_winner_same_year
0    NaN                        0                        0
1    3.0                        1                        0
2    NaN                        0                        0
3    NaN                        0                        0
4    NaN                        0                        0
5    NaN                        0                        0
6    NaN                        0                        0
7    NaN                        0                        0
8    NaN                        0                        0
```

9 NaN

0

0

Same as before but only for the one year before the world cup to look for the second Hypotheses. We want to look if the ballondor winner of the previous year has an influence on the world cup winner.

```
[177]: unique_players["year_minus_1"] = unique_players["year"] - 1

players_bd_before = unique_players.merge(
    ballondor[["Year", "player_fullname", "Rank"]],
    how="left",
    left_on=["year_minus_1", "player_fullname"],
    right_on=["Year", "player_fullname"],
    suffixes=("", "_bd_before")
)

players_bd_before["is_bd_candidate_before"] = (players_bd_before["Rank"] <= 15).
↳astype(int)
players_bd_before["is_bd_winner_before"] = (players_bd_before["Rank"] == 1).
↳astype(int)

players_bd_before.head()
```

```
[177]:
```

	year	player_id	player_fullname	team_id	team_name	size	\
0	1970	P-00086	roberto rivas	T-27	El Salvador	3	
1	1970	P-00119	gigi riva	T-41	Italy	6	
2	1970	P-00746	tom turesson	T-74	Sweden	2	
3	1970	P-01172	dobromir zhechev	T-10	Bulgaria	2	
4	1970	P-02102	anatoliy byshovets	T-72	Soviet Union	4	

	year_minus_1	Year	Rank	is_bd_candidate_before	is_bd_winner_before
0	1969	NaN	NaN	0	0
1	1969	NaN	NaN	0	0
2	1969	NaN	NaN	0	0
3	1969	NaN	NaN	0	0
4	1969	NaN	NaN	0	0

Define variable to store wc standings without duplicates

```
[178]: team_info = wc_standings[["year", "team_id", "team_name", "team_code",
↳"position"]].drop_duplicates()
team_info.head(10)
```

```
[178]:
```

	year	team_id	team_name	team_code	position
0	1970	T-09	Brazil	BRA	1
1	1970	T-41	Italy	ITA	2
2	1970	T-86	West Germany	DEU	3
3	1970	T-84	Uruguay	URY	4

4	1974	T-86	West Germany	DEU	1
5	1974	T-48	Netherlands	NLD	2
6	1974	T-57	Poland	POL	3
7	1974	T-09	Brazil	BRA	4
8	1978	T-03	Argentina	ARG	1
9	1978	T-48	Netherlands	NLD	2

Add wc team rank from team info to the player of tabel player_bd_same. We match players of their teams by year, team id und team_name. In this way we can check if the players wc rank has influence of ballondor ranking.

```
[179]: players_bd_same = players_bd_same.merge(
        team_info[["year", "team_id", "team_name", "position"]].drop_duplicates(),
        on=["year", "team_id", "team_name"],
        how="left"
    )
```

Create new column to store binary if wc winner = 1 or not = 0

```
[180]: players_bd_same["is_worldcup_winner_team"] = (players_bd_same["position"] == 1).
        ↪astype(int)
        print(players_bd_same["is_worldcup_winner_team"].head(10))
```

```
0    0
1    0
2    0
3    0
4    0
5    0
6    0
7    0
8    0
9    1
```

Name: is_worldcup_winner_team, dtype: int64

Create tabel bd_ranks. Get all players that have a Ballondor ranking in the WC year and therefore are in the Top 15. Get the variable to show if the player is a wc winner of the same year.

```
[181]: bd_ranks = players_bd_same.loc[
        players_bd_same["Rank"].notna(),
        ["Rank", "is_worldcup_winner_team"]
    ].copy()

    bd_ranks["Rank"] = bd_ranks["Rank"].astype(int)
    bd_ranks["is_worldcup_winner_team"] = bd_ranks["is_worldcup_winner_team"].
        ↪astype(int)

    players_bd_same.groupby("year")["is_bd_candidate_same_year"].sum()
```



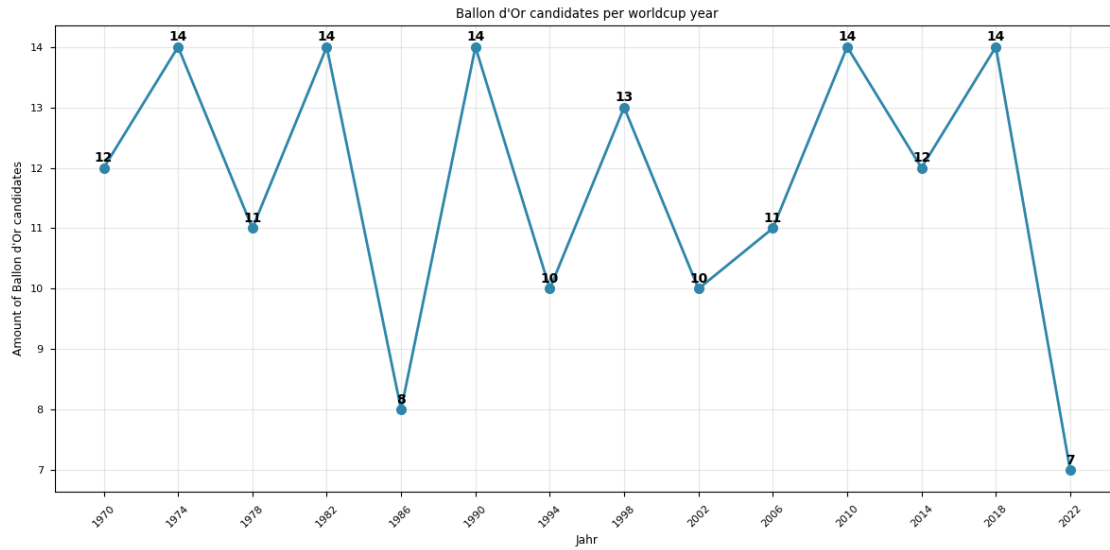
```
[181]: year
      1970    12
      1974    14
      1978    11
      1982    14
      1986     8
      1990    14
      1994    10
      1998    13
      2002    10
      2006    11
      2010    14
      2014    12
      2018    14
      2022     7
      Name: is_bd_candidate_same_year, dtype: int64
```

```
[182]: candidate_counts = players_bd_same.groupby("year")["is_bd_candidate_same_year"].
      ↪sum()

fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(candidate_counts.index, candidate_counts.values, marker='o',
        linewidth=2, markersize=8, color='#2E86AB')
ax.set_xlabel('Jahr')
ax.set_ylabel('Amount of Ballon d\'Or candidates')
ax.set_title('Ballon d\'Or candidates per worldcup year')
ax.grid(True, alpha=0.3)
ax.set_xticks(candidate_counts.index)
ax.set_xticklabels(candidate_counts.index, rotation=45)

for year, count in candidate_counts.items():
    ax.text(year, count + 0.1, str(count), ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



```
[183]: players_bd_before.groupby("year")["is_bd_candidate_before"].sum()
```

```
[183]: year
1970      0
1974     12
1978      8
1982     10
1986     10
1990     10
1994      8
1998     10
2002     11
2006      9
2010     10
2014      8
2018     12
2022      7
Name: is_bd_candidate_before, dtype: int64
```

This block aggregates player-level information into team-level statistics for each World Cup year. After merging player data with team placements, we compute:

number of players per team (“num_players”)

number and presence of Ballon d’Or candidates from the previous year

number and presence of Ballon d’Or winners from the previous year

final World Cup placement (“position”)

whether the team became World Cup champion (“is_worldcup_winner”)

```
[184]: team_summary_before = (
    players_bd_before
    .merge(team_info, on=["year", "team_id", "team_name"], how="left")
    .groupby(["year", "team_id", "team_name", "team_code"], as_index=False)
    .agg(
        num_players = ("player_id", "nunique"),
        num_bd_candidates_before = ("is_bd_candidate_before", "sum"),
        has_bd_candidate_before = ("is_bd_candidate_before", lambda x: x.
↪any()),
        num_bd_winners_before = ("is_bd_winner_before", "sum"),
        has_bd_winner_before = ("is_bd_winner_before", lambda x: x.
↪any()),
        position = ("position", "first")
    )
)

team_summary_before["is_worldcup_winner"] = (team_summary_before["position"] == 1).astype(int)
↪1).astype(int)

team_summary_before.tail()
#print(team_summary_before)
```

```
[184]:
```

	year	team_id	team_name	team_code	num_players	num_bd_candidates_before	\
51	2018	T-30	France	FRA	21	2	
52	2022	T-03	Argentina	ARG	24	1	
53	2022	T-18	Croatia	HRV	21	0	
54	2022	T-30	France	FRA	24	1	
55	2022	T-47	Morocco	MAR	25	0	

	has_bd_candidate_before	num_bd_winners_before	has_bd_winner_before	\
51	True	0	False	
52	True	1	True	
53	False	0	False	
54	True	0	False	
55	False	0	False	

	position	is_worldcup_winner
51	1.0	1
52	1.0	1
53	3.0	0
54	2.0	0
55	4.0	0

Same as done above just for Hypotheses 2:

```
[185]: team_summary_same = (
    players_bd_same
    .groupby(["year", "team_id", "team_name"], as_index=False)
    .agg(
        num_players = ("player_id", "nunique"),
        num_bd_candidates_same = ("is_bd_candidate_same_year", "sum"),
        has_bd_candidate_same = ("is_bd_candidate_same_year", lambda x: x.
↳ any()),
        num_bd_winners_same = ("is_bd_winner_same_year", "sum"),
        has_bd_winner_same = ("is_bd_winner_same_year", lambda x: x.any())
    )
)

team_summary_same = team_summary_same.merge(
    team_info[["year", "team_id", "position"]],
    on=["year", "team_id"],
    how="left"
)

team_summary_same["is_worldcup_winner"] = (team_summary_same["position"] == 1).
↳ astype(int)

team_summary_same.tail(10)
```

```
[185]:
```

	year	team_id	team_name	num_players	num_bd_candidates_same	\
358	2022	T-63	Saudi Arabia	23	0	
359	2022	T-65	Senegal	20	0	
360	2022	T-66	Serbia	22	0	
361	2022	T-71	South Korea	21	0	
362	2022	T-73	Spain	21	0	
363	2022	T-75	Switzerland	24	0	
364	2022	T-79	Tunisia	21	0	
365	2022	T-83	United States	20	0	
366	2022	T-84	Uruguay	19	0	
367	2022	T-85	Wales	18	0	

	has_bd_candidate_same	num_bd_winners_same	has_bd_winner_same	position	\
358	False	0	False	NaN	
359	False	0	False	NaN	
360	False	0	False	NaN	
361	False	0	False	NaN	
362	False	0	False	NaN	
363	False	0	False	NaN	
364	False	0	False	NaN	
365	False	0	False	NaN	
366	False	0	False	NaN	

367	False	0	False	NaN
-----	-------	---	-------	-----

	is_worldcup_winner
358	0
359	0
360	0
361	0
362	0
363	0
364	0
365	0
366	0
367	0

Groupby years to see for the years before wc how many ballondor candidates there have been and how many winners

```
[186]: team_summary_before.groupby("year")["num_bd_candidates_before",
↳ "num_bd_winners_before"].sum()
```

```
[186]:
```

	num_bd_candidates_before	num_bd_winners_before
year		
1970	0	0
1974	5	1
1978	2	0
1982	6	1
1986	5	1
1990	5	0
1994	5	1
1998	4	0
2002	1	0
2006	2	0
2010	3	0
2014	4	0
2018	5	0
2022	2	1

Groupby years to see how many ballondor candidates and winners in the same year as wc there are

```
[187]: team_summary_same.groupby("year")["num_bd_candidates_same",
↳ "num_bd_winners_same"].sum()
```

```
[187]:
```

	num_bd_candidates_same	num_bd_winners_same
year		
1970	12	1
1974	14	1
1978	11	0
1982	14	1

1986	8	0
1990	14	1
1994	10	1
1998	13	1
2002	10	0
2006	11	1
2010	14	1
2014	12	1
2018	14	1
2022	7	0

Descriptive Statistics

```
[188]: n_world_cups = team_summary_before[team_summary_before["is_worldcup_winner"] == 1].shape[0]

#time range
min_year = team_summary_before["year"].min()
max_year = team_summary_before["year"].max()

print("Dataset Overview")
print("-----")
print(f"World Cups analyzed: {n_world_cups}")
print(f"Year range: {min_year}-{max_year}")
```

Dataset Overview

World Cups analyzed: 14

Year range: 1970-2022

Kontingenztafel

```
[189]: contingency_table = pd.crosstab(
    team_summary_before["is_worldcup_winner"],
    team_summary_before["has_bd_winner_before"],
    rownames=["World Cup Winner"],
    colnames=["Has Ballon d'Or Winner Before"]
)

print("\nContingency Table")
print("-----")
print(contingency_table)
```

Contingency Table

Has Ballon d'Or Winner Before	False	True
World Cup Winner		

0	38	4
1	13	1

Ballondor Statistics

Test's

Hypothesis 1: BD candidates in the previous year → influence on WM win?

Construct a 2x2 matrix tabel to compare possibilities:

is_woldcup_winner = 1 → has won wc

has_bd_winner_before = 1 → has won bd year before wc

This shows how often the winning team has a ballondor winner in their team and how often not and of there is even a correlation. Apply Fishers Exact Test to determin the statistical significance assosiated between the 2 variables.

```
[190]: cont_before = pd.crosstab(
    team_summary_before["is_worldcup_winner"],
    team_summary_before["has_bd_winner_before"]
)

print("Contingency table: previous year's Ballon d'Or winner vs. World Cup_
↳victory:")
print(cont_before)

oddsratio_before, p_before = fisher_exact(cont_before)
print("Fisher's exact test (previous year):")
print("Odds ratio:", oddsratio_before)
print("p-value:", p_before)
```

Contingency table: previous year's Ballon d'Or winner vs. World Cup victory:

has_bd_winner_before	False	True
----------------------	-------	------

is_worldcup_winner	
--------------------	--

0	38	4
---	----	---

1	13	1
---	----	---

Fisher's exact test (previous year):

Odds ratio: 0.7307692307692307

p-value: 1.0

```
[191]: #Claud AI was used to improve the plots quality

cont_before = pd.crosstab(
    team_summary_before["is_worldcup_winner"],
    team_summary_before["has_bd_winner_before"],
    rownames=['World Cup winner'],
```

```

    colnames=['BD winner (previous year)']
)

oddsratio_before, p_before = fisher_exact(cont_before)

plt.style.use('seaborn-v0_8-paper')

fig, ax = plt.subplots(figsize=(8, 6), dpi=300)
cont_pct = cont_before.div(cont_before.sum(axis=1), axis=0) * 100
x = range(len(cont_pct.columns))
width = 0.35

bars1 = ax.bar(
    [i - width/2 for i in x],
    cont_pct.iloc[0],
    width,
    label='No World Cup win',
    color='#95a5a6',
    edgecolor='black',
    linewidth=1.5,
    alpha=0.8
)
bars2 = ax.bar(
    [i + width/2 for i in x],
    cont_pct.iloc[1],
    width,
    label='World Cup win',
    color='#3498db',
    edgecolor='black',
    linewidth=1.5,
    alpha=0.8
)

for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.text(
            bar.get_x() + bar.get_width() / 2.0,
            height,
            f'{height:.1f}%',
            ha='center',
            va='bottom',
            fontsize=10,
            fontweight='bold'
        )

ax.set_xlabel('BD winner in previous year', fontsize=14, fontweight='bold')

```



```

ax.set_ylabel('Proportion (%)', fontsize=14, fontweight='bold')
ax.set_title(
    'Percentage distribution by BD winner status',
    fontsize=16,
    fontweight='bold',
    pad=20
)
ax.set_xticks(x)
ax.set_xticklabels(cont_before.columns, fontsize=12)
ax.legend(fontsize=12, framealpha=0.9, edgecolor='black', loc='upper right')

ax.grid(axis='y', alpha=0.3, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)
ax.tick_params(axis='both', labelsize=11)

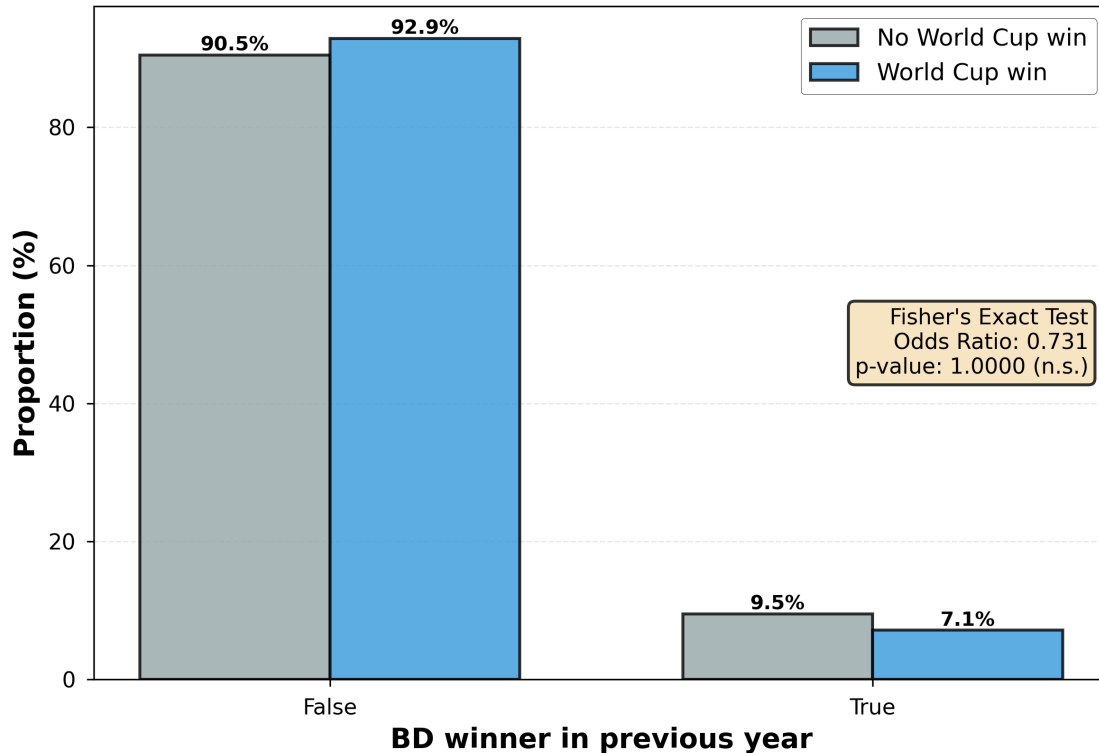
sig_text = (
    "***" if p_before < 0.001
    else "**" if p_before < 0.01
    else "*" if p_before < 0.05
    else "n.s."
)
stats_text = (
    "Fisher's Exact Test\n"
    f"Odds Ratio: {oddsratio_before:.3f}\n"
    f"p-value: {p_before:.4f} ({sig_text})"
)

ax.text(
    0.98,
    0.5,
    stats_text,
    transform=ax.transAxes,
    fontsize=11,
    verticalalignment='center',
    horizontalalignment='right',
    bbox=dict(
        boxstyle='round',
        facecolor='wheat',
        alpha=0.8,
        edgecolor='black',
        linewidth=1.5
    )
)

plt.tight_layout()
plt.show()

```

Percentage distribution by BD winner status



Hypothesis 2: WC win → Better Ballon d'Or ranking in the same year? Prep ballndor data set

First create a colum to have a binary value to check if the player is wc winner. Then extract players how have a ranking in ballondor of the year of wc. Therefor we only use two Variables “Rank” and “is_worldcup_winner”

```
[192]: players_bd_same["is_worldcup_winner_team"] = (players_bd_same["position"] == 1).
        ↳astype(int)

bd_ranks = players_bd_same.loc[
    players_bd_same["Rank"].notna(),
    ["Rank", "player_fullname", "is_worldcup_winner_team"]
].copy()

bd_ranks["Rank"] = bd_ranks["Rank"].astype(int)
bd_ranks["player_fullname"] = bd_ranks["player_fullname"].astype(str)
bd_ranks["is_worldcup_winner_team"] = bd_ranks["is_worldcup_winner_team"].
    ↳astype(int)

bd_ranks.tail(10)
```

```
#players_bd_same.head()
```

```
[192]:
```

	Rank	player_fullname	is_worldcup_winner_team
5893	6	mohamed salah	0
5982	3	antoine griezmann	1
6020	11	n'golo kanté	1
6034	4	robert lewandowski	0
6092	7	thibaut courtois	0
6208	15	rafael leão	0
6229	9	luka modrić	0
6368	3	kevin de bruyne	0
6468	6	kylian mbappé	0
6668	8	vinícius júnior	0

Splitting ballondor ranked players it the two groups where one is the winner of wc and the rest in a different group. These two distributions are required for the Mann–Whitney U test, which examines whether players from World Cup winning teams achieve significantly better/lower Ballon d’Or rankings in the same year

```
[193]: winner_ranks = bd_ranks[bd_ranks["is_worldcup_winner_team"] == 1]["Rank"]
other_ranks = bd_ranks[bd_ranks["is_worldcup_winner_team"] == 0]["Rank"]

winner_ranks.head()
```

```
[193]: 398      4
455    12
462     7
464     2
1002    8
Name: Rank, dtype: int64
```

Convert to integer

```
[194]: winner_ranks = winner_ranks.astype(int)
other_ranks = other_ranks.astype(int)
```

```
[195]: stat, p = mannwhitneyu(winner_ranks, other_ranks, alternative="two-sided")
print("Mann-Whitney U:", stat)
print("p-value:", p)
```

Mann-Whitney U: 2216.0
p-value: 0.39223829988645753

```
[196]: plt.style.use('seaborn-v0_8-paper')
fig, ax = plt.subplots(figsize=(8, 6), dpi=300)

bp = ax.boxplot([winner_ranks, other_ranks],
                labels=["Winner", "Others"],
                showmeans=True,
```

```

        meanprops=dict(marker='D', markerfacecolor='red',
                        markeredgecolor='darkred', markersize=8),
        medianprops=dict(color='darkblue', linewidth=2),
        boxprops=dict(linewidth=1.5, color='black'),
        whiskerprops=dict(linewidth=1.5, color='black'),
        capprops=dict(linewidth=1.5, color='black'),
        flierprops=dict(marker='o', markerfacecolor='gray',
                        markersize=6, alpha=0.5),
        patch_artist=True)

colors = ['#3498db', '#95a5a6'] #blue for winner, gray for others
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)

ax.set_title("Mann-Whitney U Test", fontsize=16, fontweight='bold', pad=20)
ax.set_ylabel("Rank values", fontsize=14, fontweight='bold')
ax.set_xlabel("Group", fontsize=14, fontweight='bold')

#textbox for the p-value
ax.text(0.98, 0.97, 'p = 0.39',
        transform=ax.transAxes,
        fontsize=12,
        verticalalignment='top',
        horizontalalignment='right',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

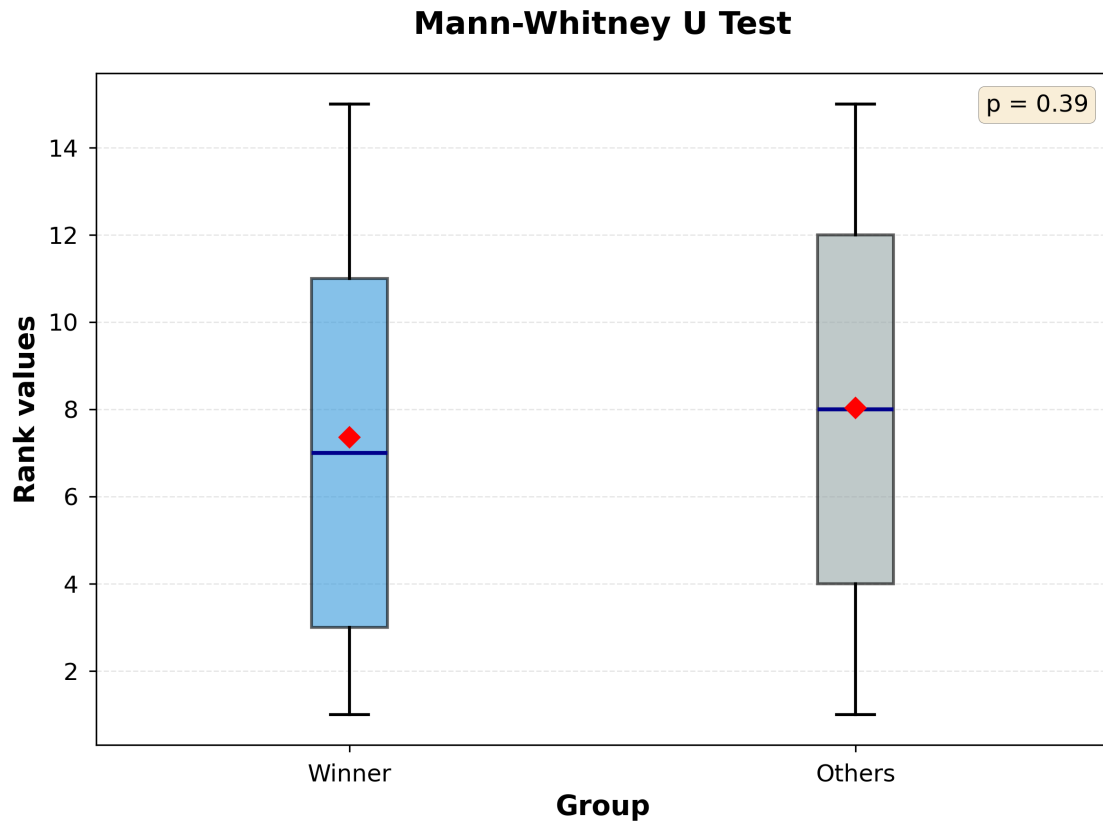
ax.grid(axis="y", alpha=0.3, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)
ax.tick_params(axis='both', which='major', labelsize=12)

plt.tight_layout()
#safe as picture and pdf for paper
plt.savefig('mann_whitney_test.png', dpi=300, bbox_inches='tight')
plt.savefig('mann_whitney_test.pdf', bbox_inches='tight')
plt.show()

```

C:\Users\PC\AppData\Local\Temp\ipykernel_24612\601641511.py:4:
MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

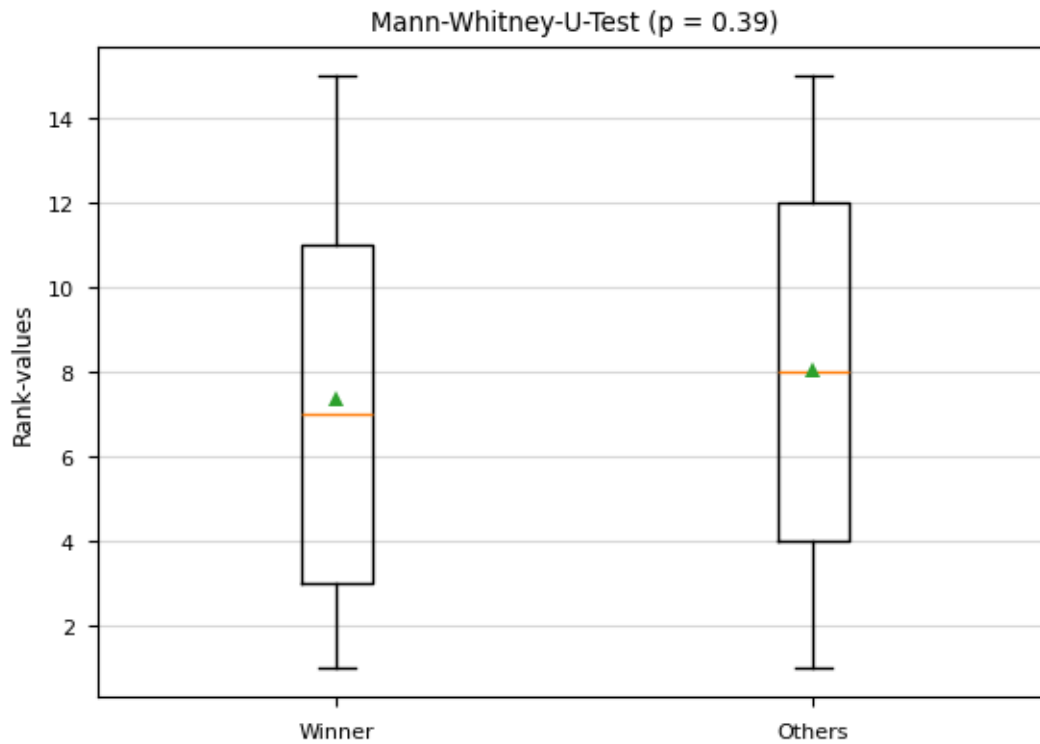
```
bp = ax.boxplot([winner_ranks, other_ranks],
```



```
[197]: plt.boxplot([winner_ranks, other_ranks],
                labels=["Winner", "Others"],
                showmeans=True)

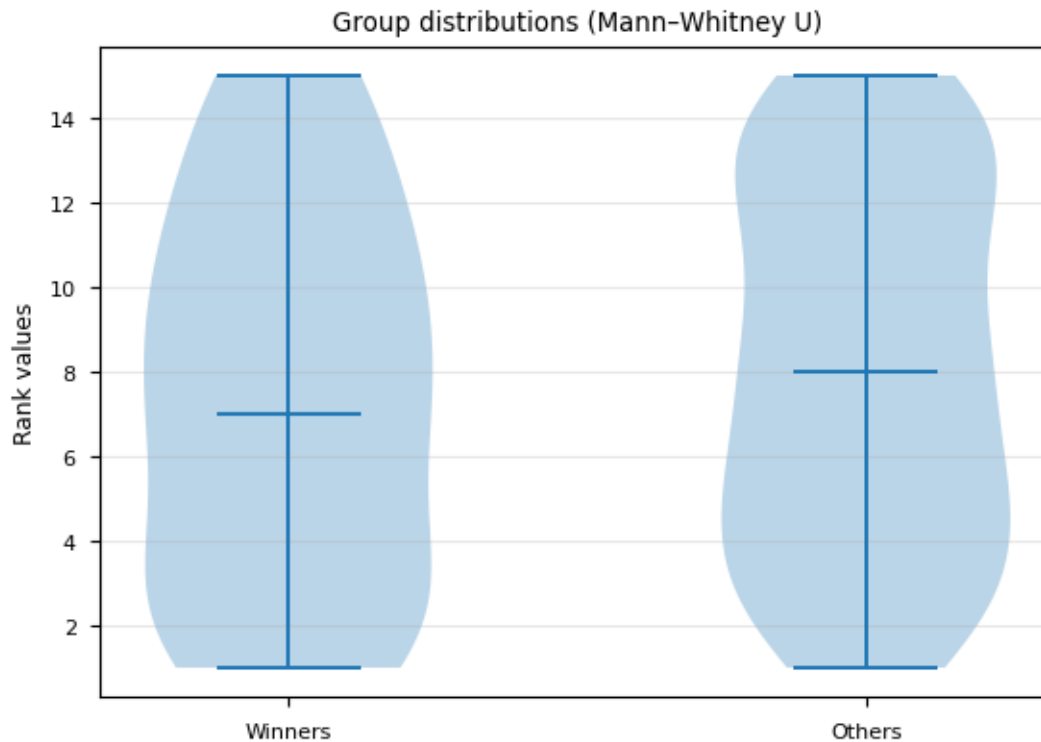
plt.title("Mann-Whitney-U-Test (p = 0.39)")
plt.ylabel("Rank-values")
plt.grid(axis="y", alpha=0.5)
plt.show()
```

```
C:\Users\PC\AppData\Local\Temp\ipykernel_24612\2370735703.py:1:
MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
plt.boxplot([winner_ranks, other_ranks],
```



```
[198]: plt.violinplot([winner_ranks, other_ranks], showmedians=True)

plt.xticks([1, 2], ["Winners", "Others"])
plt.title("Group distributions (Mann-Whitney U)")
plt.ylabel("Rank values")
plt.grid(axis="y", alpha=0.3)
plt.show()
```



```
[223]: plt.style.use('seaborn-v0_8-paper') # [web:19]
fig, ax = plt.subplots(figsize=(8, 6), dpi=300)

parts = ax.violinplot([winner_ranks, other_ranks],
                      showmedians=True,
                      showextrema=True,
                      showmeans=True)

colors = ['#3498db', '#95a5a6']
for i, pc in enumerate(parts['bodies']):
    pc.set_facecolor(colors[i])
    pc.set_alpha(0.6)
    pc.set_edgecolor('black')
    pc.set_linewidth(1.5)

parts['cmedians'].set_edgecolor('darkblue')
parts['cmedians'].set_linewidth(2.5)

parts['cmaxes'].set_edgecolor('black')
parts['cmaxes'].set_linewidth(1.5)
parts['cmins'].set_edgecolor('black')
```

```

parts['cmins'].set_linewidth(1.5)
parts['cbars'].set_edgecolor('black')
parts['cbars'].set_linewidth(1.5)

parts['cmeans'].set_edgecolor('red')
parts['cmeans'].set_linewidth(2)
parts['cmeans'].set_linestyle('--')

ax.set_xticks([1, 2])
ax.set_xticklabels(["Winner", "Others"], fontsize=14, fontweight='bold')
ax.set_ylabel("Rank values", fontsize=14, fontweight='bold')
ax.set_xlabel("Group", fontsize=14, fontweight='bold')

ax.set_title("Group distributions", fontsize=16, fontweight='bold', pad=20)
ax.text(0.98, 0.97, 'Mann-Whitney U\ntp = 0.39',
        transform=ax.transAxes,
        fontsize=11,
        verticalalignment='top',
        horizontalalignment='right',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

legend_elements = [
    Line2D([0], [0], color='darkblue', lw=2.5, label='Median'),
    Line2D([0], [0], color='red', lw=2, linestyle='--', label='Mean')
]
ax.legend(handles=legend_elements, loc='upper left', fontsize=11, framealpha=0.
↪9)

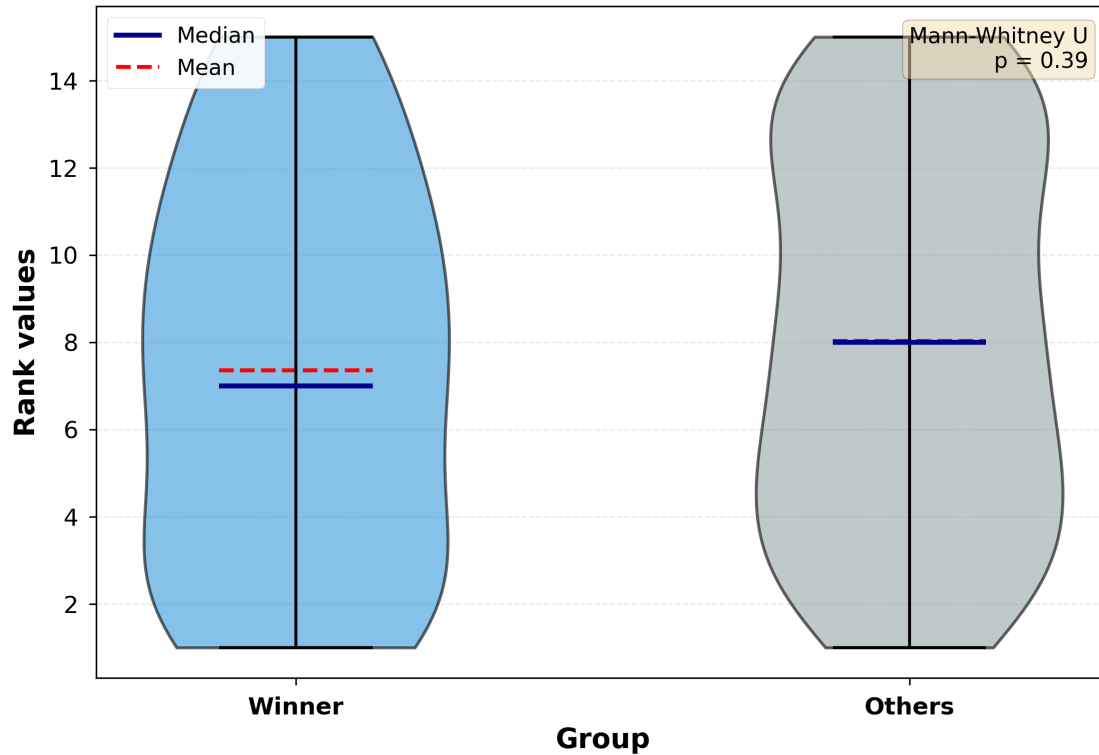
ax.grid(axis="y", alpha=0.3, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)
ax.tick_params(axis='both', which='major', labelsize=12)

plt.tight_layout()

#output png and pdf for paper
plt.savefig('mann_whitney_violin.png', dpi=300, bbox_inches='tight')
plt.savefig('mann_whitney_violin.pdf', bbox_inches='tight')
plt.show()

```


Group distributions



Permutationtest

```
[200]: observed_diff = (
    bd_ranks.loc[bd_ranks["is_worldcup_winner_team"] == 1, "Rank"].median()
    - bd_ranks.loc[bd_ranks["is_worldcup_winner_team"] == 0, "Rank"].median()
)

print(f"Observed difference between both medians is: {observed_diff}")
```

Observed difference between both medians is: -1.0

```
[201]: import numpy as np

n_perm = 10000
perm_diffs = []

for _ in range(n_perm):
    shuffled = bd_ranks["is_worldcup_winner_team"].sample(frac=1,
        replace=False).values
    perm_diff = (
```

```

        bd_ranks.loc[shuffled == 1, "Rank"].median()
        - bd_ranks.loc[shuffled == 0, "Rank"].median()
    )
    perm_diffs.append(perm_diff)

perm_diffs = np.array(perm_diffs)

```

```

[202]: plt.style.use('seaborn-v0_8-paper')
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

n, bins, patches = ax.hist(
    perm_diffs,
    bins=30,
    density=True,
    alpha=0.7,
    color='#3498db',
    edgecolor='black',
    linewidth=1.2,
    label='Permutation distribution'
)

p_value = np.mean(np.abs(perm_diffs) >= np.abs(observed_diff))

ax.axvline(
    observed_diff,
    color='#e74c3c',
    linestyle='--',
    linewidth=3,
    label=f'Observed difference = {observed_diff:.3f}'
)

alpha_level = 0.05
lower_crit = np.percentile(perm_diffs, alpha_level / 2 * 100)
upper_crit = np.percentile(perm_diffs, (1 - alpha_level / 2) * 100)

ax.axvline(lower_crit, color='gray', linestyle=':', linewidth=2, alpha=0.7)
ax.axvline(
    upper_crit,
    color='gray',
    linestyle=':',
    linewidth=2,
    alpha=0.7,
    label=f'Critical values ( = {alpha_level})'
)

ax.axvspan(perm_diffs.min(), lower_crit, alpha=0.2, color='red')

```

```

ax.axvspan(upper_crit, perm_diffs.max(), alpha=0.2, color='red') # [web:29]
ax.set_xlabel(
    "Median difference in Ballon d'Or rank",
    fontsize=14,
    fontweight='bold'
)
ax.set_ylabel("Density", fontsize=14, fontweight='bold')
ax.set_title(
    "Permutation test: median rank difference\n(World Cup winners vs. other_
→teams)",
    fontsize=16,
    fontweight='bold',
    pad=20
)

ax.legend(fontsize=11, framealpha=0.9, edgecolor='black', loc='upper right')
ax.grid(axis='y', alpha=0.3, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)
ax.tick_params(axis='both', which='major', labelsize=12)

stats_text = (
    f"Permutation test\n"
    f"Observed difference: {observed_diff:.3f}\n"
    f"p-value: {p_value:.4f}"
)

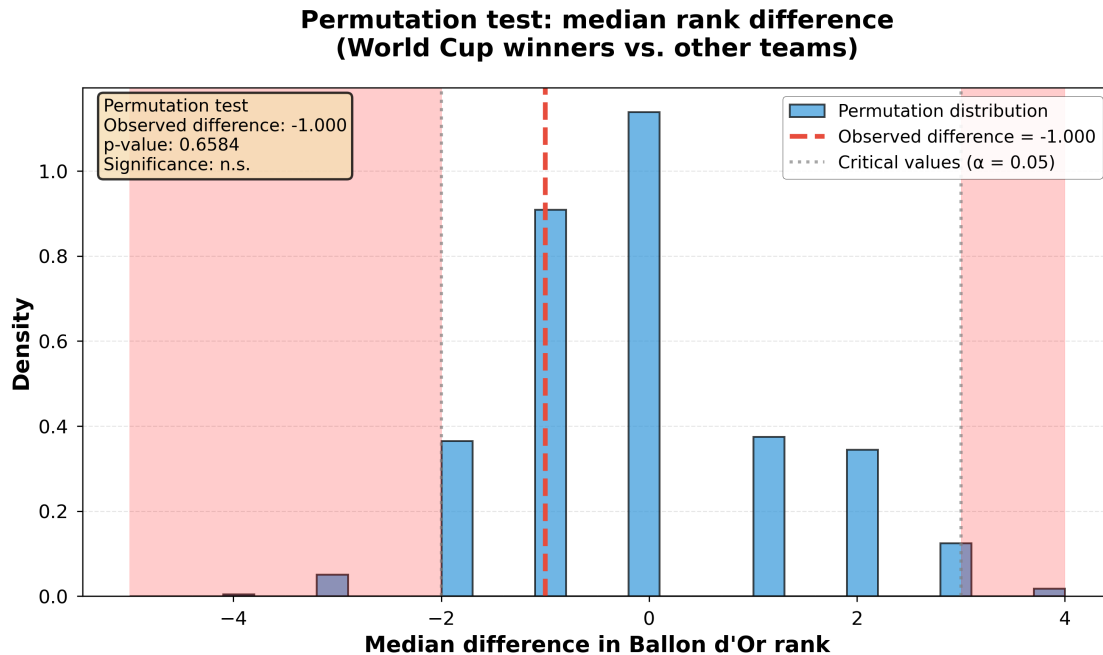
sig_text = (
    "***" if p_value < 0.001
    else "**" if p_value < 0.01
    else "*" if p_value < 0.05
    else "n.s."
)

ax.text(
    0.02,
    0.98,
    stats_text + f"\nSignificance: {sig_text}",
    transform=ax.transAxes,
    fontsize=11,
    verticalalignment='top',
    bbox=dict(
        boxstyle='round',
        facecolor='wheat',
        alpha=0.8,
        edgecolor='black',
        linewidth=1.5
    )
)

```

```
)

plt.tight_layout()
plt.show()
```



Cliffs Delta for the effect size.

```
[203]: def cliffs_delta(x, y):
        gt = sum(i > j for i in x for j in y)
        lt = sum(i < j for i in x for j in y)
        return (gt - lt) / (len(x) * len(y))

x = winner_ranks.to_numpy()
y = other_ranks.to_numpy()

delta = cliffs_delta(x, y)
print("Delta value:", delta)
```

Delta value: -0.09087179487179488

```
[204]: def bootstrap_cliffs_delta(x, y, n_boot=10000, seed=42):
        rng = np.random.default_rng(seed)
        deltas = np.empty(n_boot)
        for b in range(n_boot):
            xb = rng.choice(x, size=len(x), replace=True)
```

```

        yb = rng.choice(y, size=len(y), replace=True)
        deltas[b] = cliffs_delta(xb, yb)
    ci_low, ci_high = np.percentile(deltas, [2.5, 97.5])
    return ci_low, ci_high

ci_low, ci_high = bootstrap_cliffs_delta(x, y, n_boot=10000, seed=42)
print(f"Lower bound: {ci_low}, Higher bound {ci_high}")

```

Lower bound: -0.3021589743589744, Higher bound 0.11898461538461523

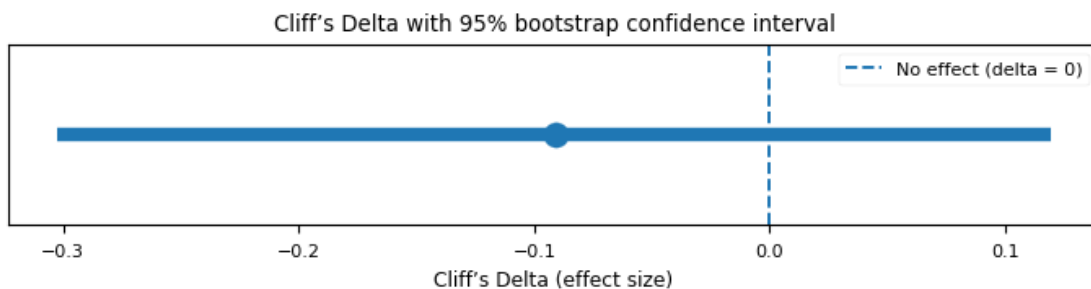
```

[205]: fig, ax = plt.subplots(figsize=(7, 2))

ax.hlines(y=0, xmin=ci_low, xmax=ci_high, linewidth=6)
ax.scatter(delta, 0, s=120, zorder=3)
ax.axvline(0, linestyle="--", label="No effect (delta = 0)")

ax.set_yticks([])
ax.set_xlabel("Cliff's Delta (effect size)")
ax.set_title("Cliff's Delta with 95% bootstrap confidence interval")
ax.legend()
plt.tight_layout()
plt.show()

```



```

[206]: plt.style.use('seaborn-v0_8-paper')
fig, ax = plt.subplots(figsize=(10, 3), dpi=300)

ax.hlines(
    y=0,
    xmin=ci_low,
    xmax=ci_high,
    linewidth=12,
    color='#3498db',
    alpha=0.7,
    capstyle='round'
)

```

```

ax.scatter(
    [ci_low, ci_high],
    [0, 0],
    s=100,
    color='#3498db',
    edgecolors='darkblue',
    linewidth=2,
    zorder=3
)

ax.scatter(
    delta,
    0,
    s=250,
    color='#e74c3c',
    marker='D',
    edgecolors='darkred',
    linewidth=2.5,
    zorder=4
)

ax.axvline(0, linestyle='--', color='black', linewidth=2.5, alpha=0.7)

ax.text(
    delta,
    0.15,
    f" = {delta:.3f}",
    ha='center',
    fontsize=12,
    fontweight='bold',
    bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8)
)

ax.text(
    0,
    -0.15,
    "No effect",
    ha='center',
    fontsize=10,
    style='italic'
)

ax.set_yticks([])
ax.set_xlabel("Cliff's delta (effect size)", fontsize=14, fontweight='bold')
ax.set_xlim(-1, 1)
ax.set_ylim(-0.3, 0.3)

ax.set_title(

```

```

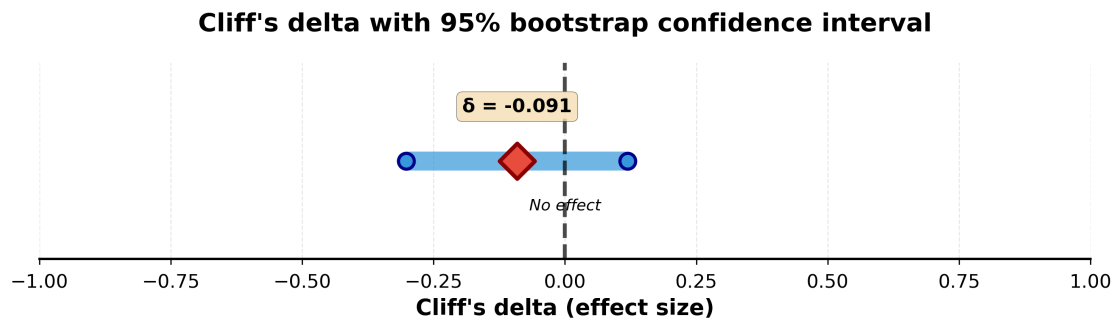
    "Cliff's delta with 95% bootstrap confidence interval",
    fontsize=16,
    fontweight='bold',
    pad=20
)

ax.grid(axis='x', alpha=0.3, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)

ax.spines['left'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['top'].set_visible(False)
ax.spines['bottom'].set_linewidth(1.5)
ax.tick_params(axis='x', which='major', labelsize=12)

plt.tight_layout()
plt.savefig('cliffs_delta_ci.png', dpi=300, bbox_inches='tight')
plt.savefig('cliffs_delta_ci.pdf', bbox_inches='tight')
plt.show()

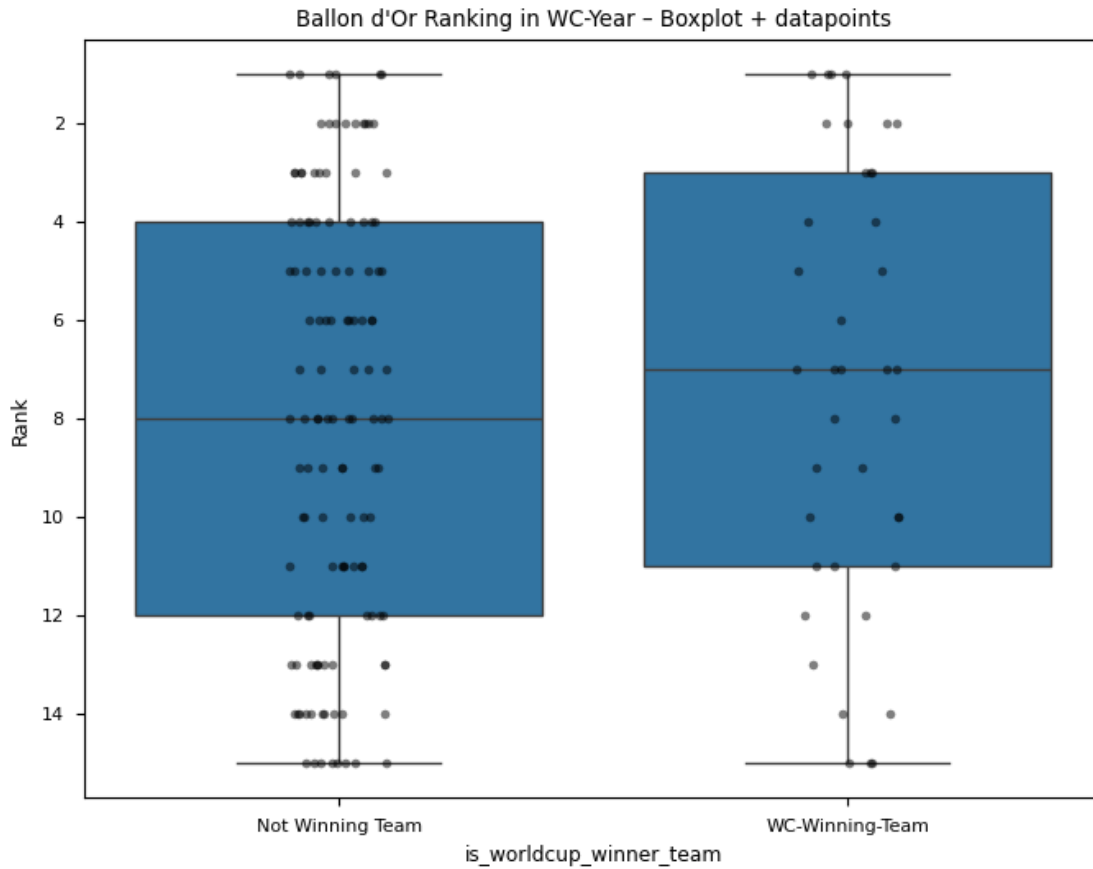
```



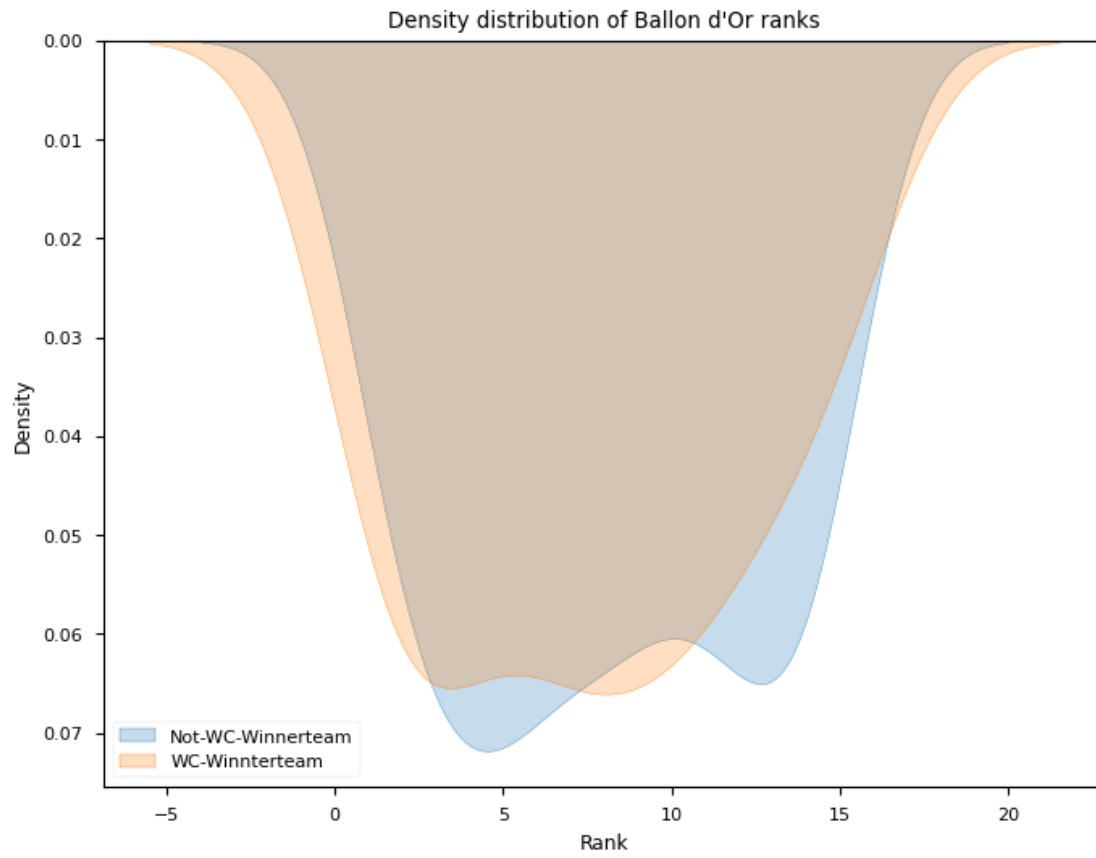
```

[207]: plt.figure(figsize=(8,6))
sns.boxplot(data=bd_ranks, x="is_worldcup_winner_team", y="Rank",
            ↪showfliers=False)
sns.stripplot(data=bd_ranks, x="is_worldcup_winner_team", y="Rank",
            ↪color="black", size=4, alpha=0.5)
plt.gca().invert_yaxis()
plt.xticks([0, 1], ["Not Winning Team", "WC-Winning-Team"])
plt.title("Ballon d'Or Ranking in WC-Year - Boxplot + datapoints")
plt.show()

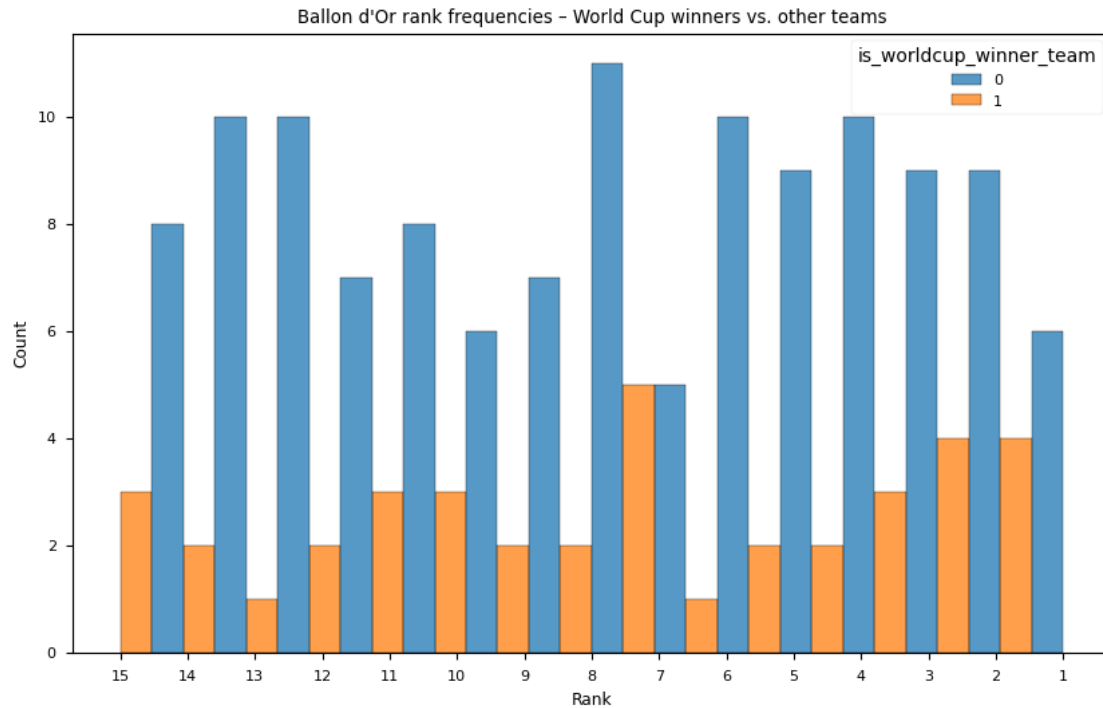
```



```
[208]: plt.figure(figsize=(8,6))
sns.kdeplot(data=bd_ranks[bd_ranks["is_worldcup_winner_team"] == 0]["Rank"],
            label="Not-WC-Winnerteam", fill=True)
sns.kdeplot(data=bd_ranks[bd_ranks["is_worldcup_winner_team"] == 1]["Rank"],
            label="WC-Winnterteam", fill=True)
plt.gca().invert_yaxis()
plt.title("Density distribution of Ballon d'Or ranks")
plt.legend()
plt.show()
```

```
[209]: plt.figure(figsize=(10, 6))
sns.histplot(
    bd_ranks,
    x="Rank",
    hue="is_worldcup_winner_team",
    multiple="dodge",
    bins=15
)
plt.gca().invert_xaxis()
plt.title("Ballon d'Or rank frequencies - World Cup winners vs. other teams")
plt.xticks(range(1, 16))
plt.show()
```



Hypotheses 1

Fischers Exact test for the Top 3 instad only looking for the Ballondor Winner

```
[210]: players_bd_same["is_top3"] = (players_bd_same["Rank"] <= 3).astype(int)

[211]: team_top3 = (
    players_bd_same
    .groupby(["year", "team_id", "team_name"], as_index=False)
    .agg(has_top3_player=("is_top3", "any"))
    .merge(team_info[["year", "team_id", "position"]], on=["year", "team_id"])
)

team_top3["is_worldcup_winner"] = (team_top3["position"] == 1).astype(int)

[212]: cont_top3 = pd.crosstab(
    team_top3["is_worldcup_winner"],
    team_top3["has_top3_player"]
)

odds_top3, p_top3 = fisher_exact(cont_top3)
print(f"p-value: {p_top3}")
```

```
print(odds_top3)
print(cont_top3)
```

p-value: 0.11712917019150512

3.24

has_top3_player	False	True
is_worldcup_winner		
0	27	15
1	5	9

```
[213]: mosaic_data = {
    ("WC Winner", "Top-3 Player"): 9,
    ("WC Winner", "No Top-3 Player"): 5,
    ("Not WC Winner", "Top-3 Player"): 15,
    ("Not WC Winner", "No Top-3 Player"): 27,
}
```

```
fig, ax = plt.subplots(figsize=(7,5))
```

```
mosaic(mosaic_data, ax=ax)
```

```
ax.set_title("World Cup success vs. Top-3 Ballon d'Or players")
```

```
plt.show()
```



Top 10

```
[214]: players_bd_same["is_top10"] = (players_bd_same["Rank"] <= 10).astype(int)
```

```
[215]: team_top10 = (
    players_bd_same
    .groupby(["year", "team_id", "team_name"], as_index=False)
    .agg(has_top10_player=("is_top10", "any"))
    .merge(team_info[["year", "team_id", "position"]], on=["year", "team_id"])
)

team_top10["is_worldcup_winner"] = (team_top10["position"] == 1).astype(int)
```

```
[216]: cont_top10 = pd.crosstab(
    team_top10["is_worldcup_winner"],
    team_top10["has_top10_player"]
)

odds_top10, p_top10 = fisher_exact(cont_top10)
print(f"p-vlaue: {p_top10}")
print(odds_top10)
print(cont_top10)
```

p-vlaue: 0.7507955057389036

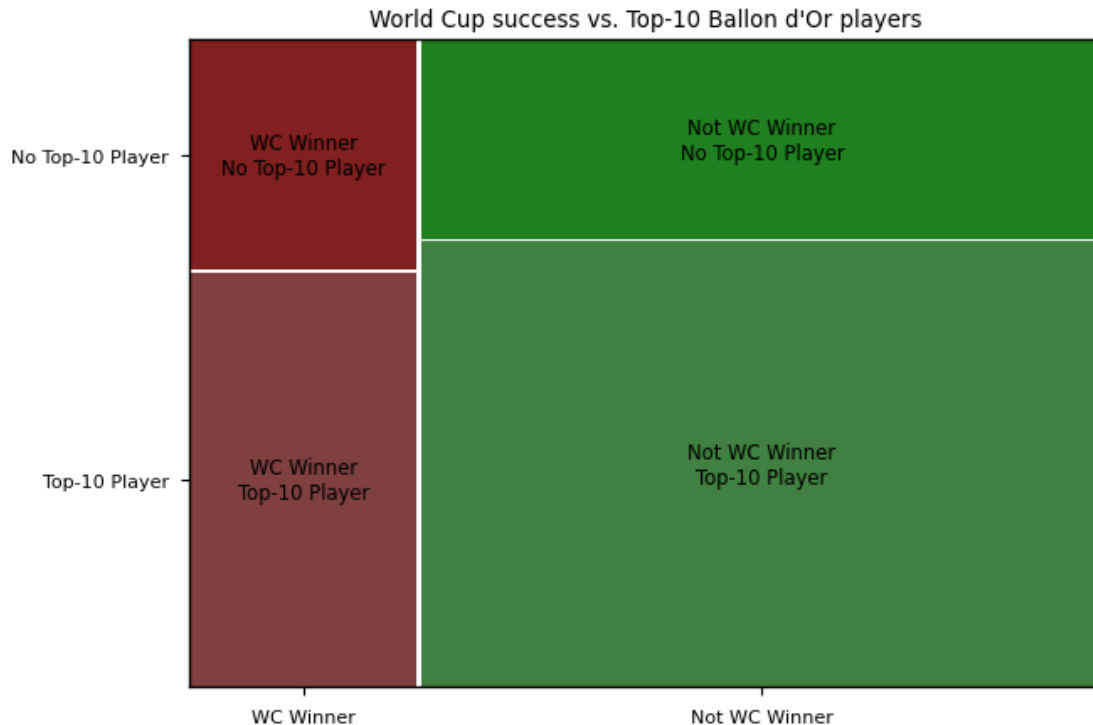
0.8068965517241379

has_top10_player	False	True
is_worldcup_winner		
0	13	29
1	5	9

```
[217]: mosaic_data = {
    ("WC Winner", "Top-10 Player"): 9,
    ("WC Winner", "No Top-10 Player"): 5,
    ("Not WC Winner", "Top-10 Player"): 29,
    ("Not WC Winner", "No Top-10 Player"): 13,
}

fig, ax = plt.subplots(figsize=(7,5))
mosaic(mosaic_data, ax=ax)

ax.set_title("World Cup success vs. Top-10 Ballon d'Or players")
plt.show()
```



Top 5

```
[218]: players_bd_same["is_top5"] = (players_bd_same["Rank"] <= 5).astype(int)
```

```
[219]: team_top5 = (
    players_bd_same
    .groupby(["year", "team_id", "team_name"], as_index=False)
    .agg(has_top5_player=("is_top5", "any"))
    .merge(team_info[["year", "team_id", "position"]], on=["year", "team_id"])
)

team_top5["is_worldcup_winner"] = (team_top5["position"] == 1).astype(int)
```

```
[220]: cont_top5 = pd.crosstab(
    team_top5["is_worldcup_winner"],
    team_top5["has_top5_player"]
)

odds_top5, p_top5 = fisher_exact(cont_top5)
print(f"p-value: {p_top5}")
print(odds_top5)
print(cont_top5)
```

```

p-value: 0.5415585221513101
1.6363636363636365
has_top5_player    False   True
is_worldcup_winner
0                   20     22
1                   5      9

```

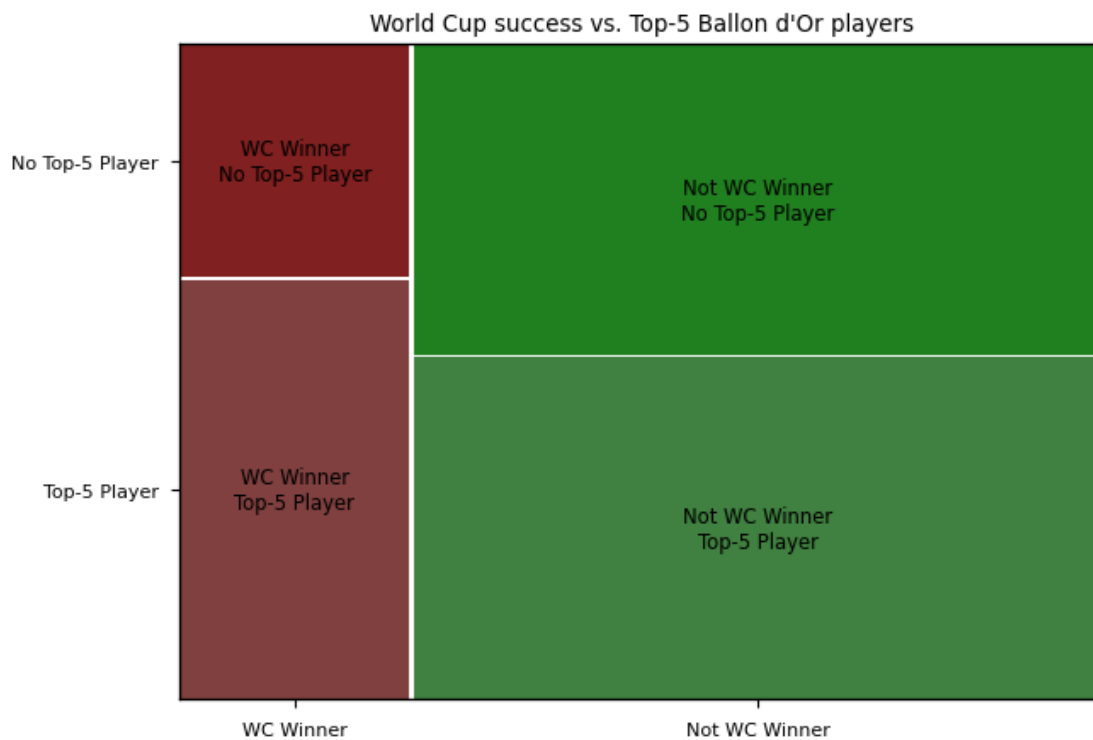
```

[221]: mosaic_data = {
    ("WC Winner", "Top-5 Player"): 9,
    ("WC Winner", "No Top-5 Player"): 5,
    ("Not WC Winner", "Top-5 Player"): 22,
    ("Not WC Winner", "No Top-5 Player"): 20,
}

fig, ax = plt.subplots(figsize=(7,5))
mosaic(mosaic_data, ax=ax)

ax.set_title("World Cup success vs. Top-5 Ballon d'Or players")
plt.show()

```



Top 15

```
[224]: players_bd_same["is_top15"] = (players_bd_same["Rank"] <= 15).astype(int)
```

```
[226]: team_top15 = (
    players_bd_same
    .groupby(["year", "team_id", "team_name"], as_index=False)
    .agg(has_top15_player=("is_top15", "any"))
    .merge(team_info[["year", "team_id", "position"]], on=["year", "team_id"])
)

team_top15["is_worldcup_winner"] = (team_top15["position"] == 1).astype(int)
```

```
[227]: cont_top15 = pd.crosstab(
    team_top15["is_worldcup_winner"],
    team_top15["has_top15_player"]
)

odds_top15, p_top15 = fisher_exact(cont_top15)
print(f"p-value: {p_top15}")
print(odds_top15)
print(cont_top15)
```

p-value: 0.3044765169480779

0.4909090909090909

has_top15_player	False	True
is_worldcup_winner		
0	9	33
1	5	9

```
[228]: mosaic_data = {
    ("WC Winner", "Top-5 Player"): 33,
    ("WC Winner", "No Top-5 Player"): 5,
    ("Not WC Winner", "Top-5 Player"): 9,
    ("Not WC Winner", "No Top-5 Player"): 9,
}

fig, ax = plt.subplots(figsize=(7,5))
mosaic(mosaic_data, ax=ax)

ax.set_title("World Cup success vs. Top-15 Ballon d'Or players")
plt.show()
```

